Rapport : Sécurité des Applications Web
Anas TAGUI

Introduction

Dans le cadre de mes etudes en securite des systemes d'information, j'ai realise une serie de travaux pratiques portant sur les vulnerabilites les plus courantes des applications web. Ces labs ont ete effectues sur deux plateformes reconnues dans le domaine : DVWA (Damn Vulnerable Web Application) et PortSwigger Web Security Academy, en utilisant Burp Suite Community Edition comme outil principal d'interception et d'analyse.

L'objectif de ces exercices n'etait pas seulement de comprendre comment une attaque fonctionne techniquement, mais surtout de comprendre pourquoi ces vulnerabilites existent, et comment les prevenir concretement dans un contexte de developpement reel.

Ce rapport est structure en trois parties :

• **Partie 1** : Principes de codage securise, a travers l'etude du CSRF sur DVWA a trois niveaux de securite progressifs.

• **Partie 2** : Authentification et gestion de session, avec des labs PortSwigger couvrant l'enumeration d'utilisateurs, le contournement de la 2FA et les failles JWT.

• **Partie 3** : Autorisation et controle d'acces, explorant les vulnerabilites IDOR, l'elevation de privileges et les fonctions d'administration non protegees.

Partie 1 : Principes de codage sécurisé (DVWA)

Vulnérabilité étudiée : Cross-Site Request Forgery (CSRF)

Outil utilisé : Damn Vulnerable Web App (DVWA)

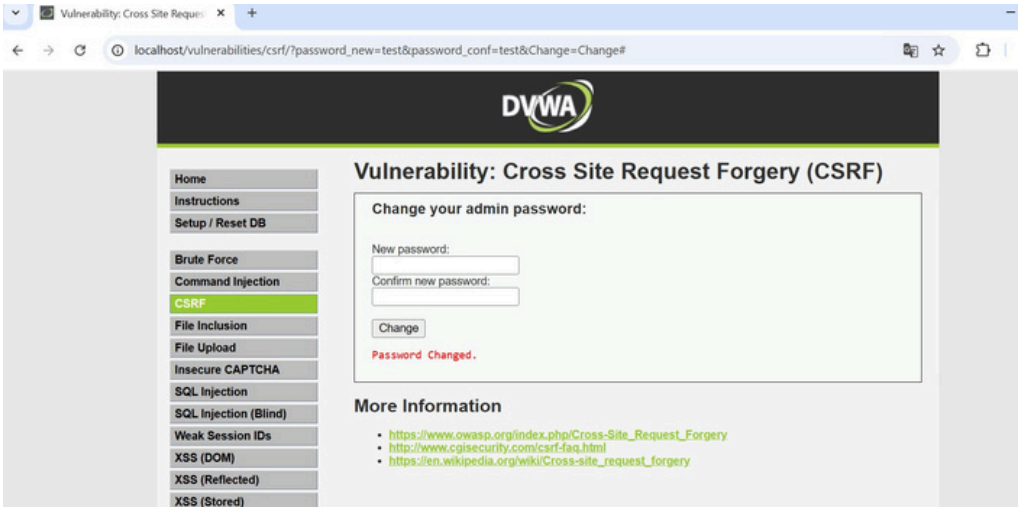
Objectif : Démontrer comment un attaquant peut forcer un utilisateur authentifié (admin) à changer son mot de passe sans son consentement, et comment contourner les mesures de sécurité à différents niveaux (Low, Medium, High).

---

1. Niveau de sécurité : Faible (Low)

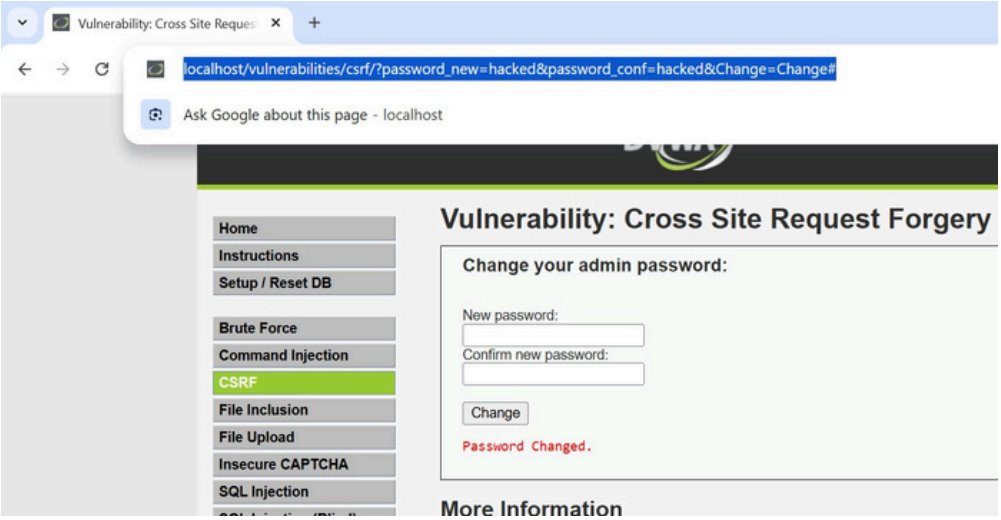
Analyse de la vulnérabilité : Au niveau "Low", l'application ne vérifie pas la source de la requête et n'utilise aucun jeton de protection. Le changement de mot de passe s'effectue via une simple requête HTTP GET prenant les paramètres password_new et `password_conf`.

Méthode d'exploitation : Il suffit de faire visiter à la victime une URL spécialement conçue contenant les paramètres du nouveau mot de passe. Si la victime est connectée, l'action est exécutée immédiatement.



Payload (URL malveillante) :
http://localhost/vulnerabilities/csrf/?password_new=hacked&password_conf=hacked&Change=Change

Preuve de concept : Nous avons entré l'URL ci-dessus dans un nouvel onglet du navigateur. Le mot de passe a été modifié instantanément.



---

2. Niveau de sécurité : Moyen (Medium)

Analyse de la vulnérabilité : Au niveau "Medium", l'application tente de se protéger en vérifiant l'en-tête HTTP Referer. Le code vérifie si le nom du serveur (ici localhost) est présent dans l'origine de la requête. Cependant, la vérification est imparfaite (utilisation de stripos) : elle vérifie seulement si le mot clé apparaît *quelque part* dans l'URL référente, et non si elle *commence* par le domaine légitime.

Méthode d'exploitation : Pour contourner cette protection, nous avons créé un fichier HTML malveillant nommé localhost.html. En nommant le fichier ainsi, l'en-tête Referer envoyé lors de l'attaque contient le mot "localhost", ce qui suffit à tromper le filtre de sécurité.

Code de l'attaque (`localhost.html`) :

localhost.html

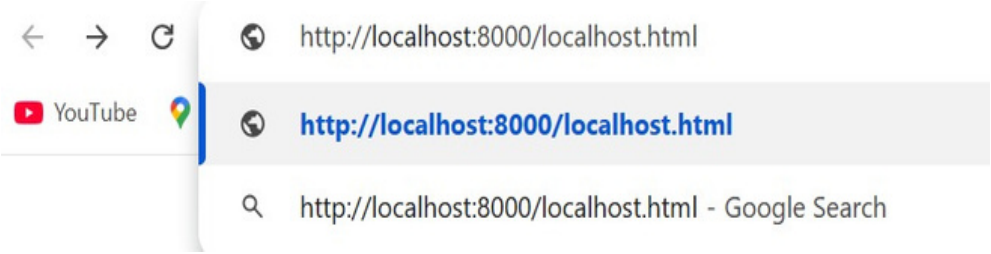
C: > Users > HP > Desktop > Documents > localhost.html

```html
<html>
  <body onload="document.forms[0].submit()">
    <form action="http://localhost/vulnerabilities/csrf/" method="GET">
      <input type="hidden" name="password_new" value="mediumhacked">
      <input type="hidden" name="password_conf" value="mediumhacked">
      <input type="hidden" name="Change" value="Change">
    </form>
  </body>
</html>
```

Preuve de concept : Le fichier a été hébergé localement (`python3 -m http.server`) et ouvert dans le navigateur.

```
PS C:\Users\HP\Desktop\Documents> python3 -m http.server 8000
Serving HTTP on :: port 8000 (http://[::]:8000/) ...
::1 - - [15/Dec/2025 14:12:18] "GET /localhost.html HTTP/1.1" 200 -
::1 - - [15/Dec/2025 14:12:19] code 404, message File not found
::1 - - [15/Dec/2025 14:12:19] "GET /favicon.ico HTTP/1.1" 404 -
```

La redirection a fonctionné et le filtre a été contourné.



---

3. Niveau de sécurité : Élevé (High)

Analyse de la vulnérabilité : Au niveau "High", l'application implémente une protection robuste : un jeton Anti-CSRF (user_token). Ce jeton est aléatoire et change à chaque chargement de page. Une attaque CSRF classique échoue car l'attaquant ne peut pas deviner ce jeton.

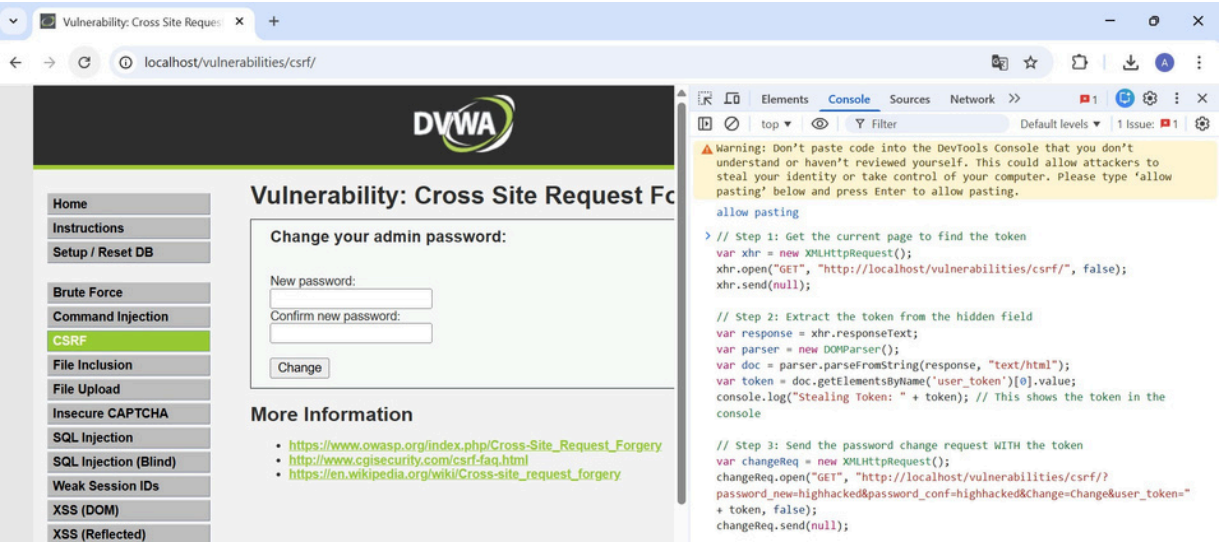
Méthode d'exploitation : Pour contourner cela, nous devons combiner le CSRF avec une exécution de script (simulant une faille XSS). Le script doit :

1. Faire une requête silencieuse (AJAX) vers la page CSRF pour récupérer le jeton valide.
2. Extraire ce jeton du code HTML reçu.
3. Envoyer la requête de changement de mot de passe en incluant ce jeton valide.

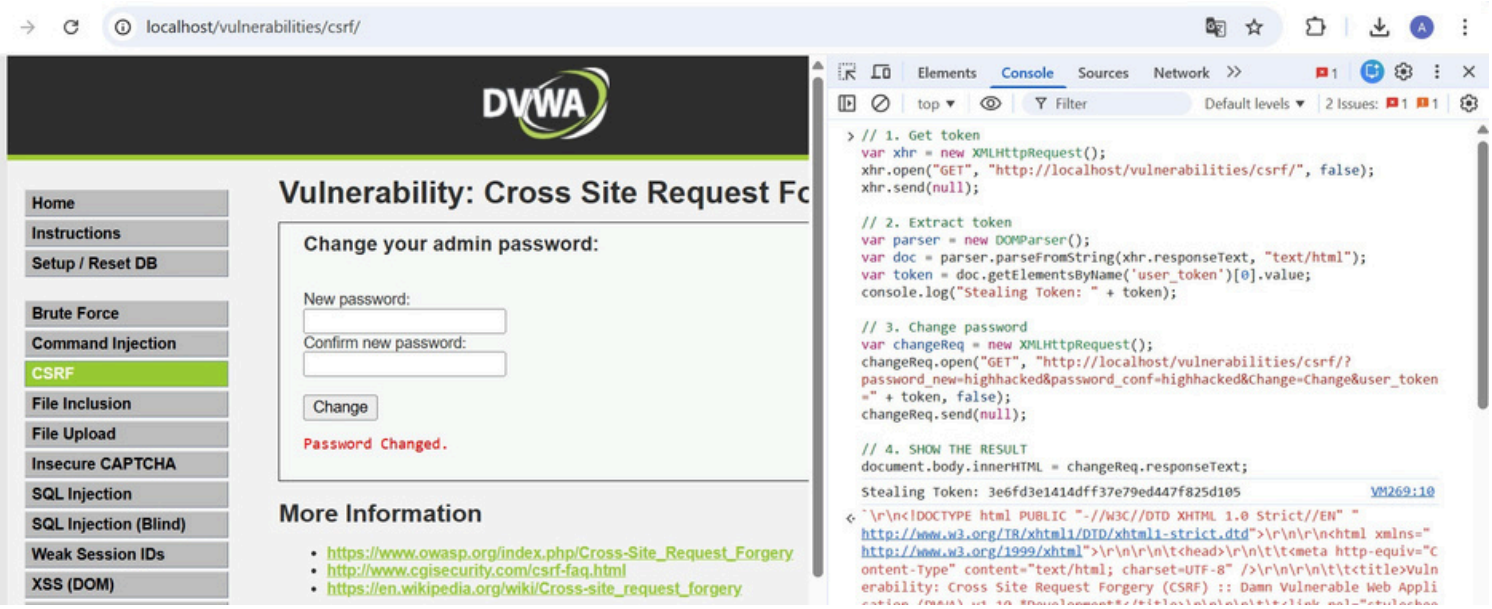
Code de l'attaque (JavaScript exécuté en console) :

JavaScript

```
// 1. Récupération du token via une requête GET
var xhr = new XMLHttpRequest();
xhr.open("GET", "http://localhost/vulnerabilities/csrf/", false);
xhr.send(null);

// 2. Extraction du token depuis le DOM
var parser = new DOMParser();
var doc = parser.parseFromString(xhr.responseText, "text/html");
var token = doc.getElementsByName('user_token')[0].value;
console.log("Token volé : " + token);

// 3. Envoi de la requête forgée avec le token valide
var changeReq = new XMLHttpRequest();
changeReq.open("GET",
"http://localhost/vulnerabilities/csrf/?password_new=highhacked&password_conf=highhacked&Change=Change&user_token=" + token, false);
changeReq.send(null);

// 4. Affichage du résultat pour confirmation visuelle
document.body.innerHTML = changeReq.responseText;
```



Preuvedeconcept : Le script a permis de récupérer le token dynamiquement et de valider la requête.



Partie 2 : Authentification et Gestion de Session (PortSwigger)

Plateforme utilisée : PortSwigger Web Security Academy

Outil utilisé : BurpSuite Community Edition

---

1. Username enumeration via different responses

Lien du lab : Objectif : Identifier un nom d'utilisateur valide parmi une liste, puis trouver son mot de passe pour se connecter.

Analyse de la vulnérabilité : L'application répond de manière légèrement différente selon que le nom d'utilisateur existe ou non. Bien que le code de statut soit toujours 200 OK, la longueur de la réponse (ou le message d'erreur spécifique) varie, ce qui permet l'énumération des utilisateurs.

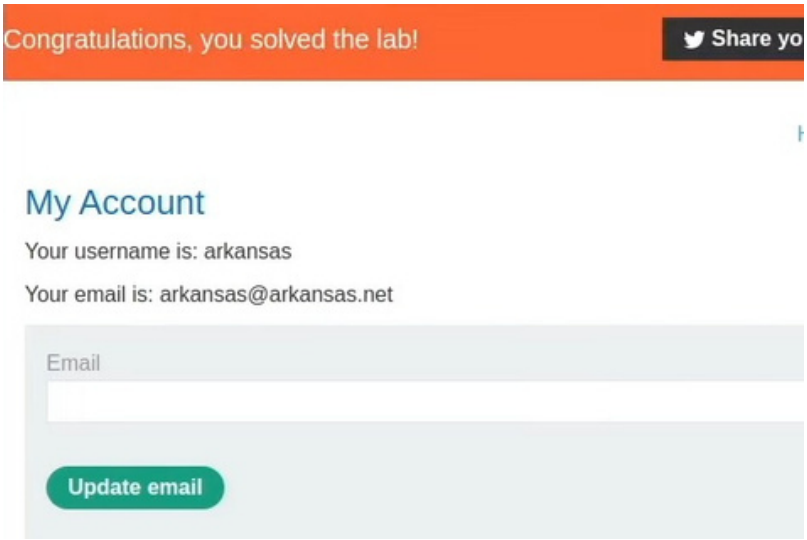
Méthode d'exploitation :

1. **Énumération de l'utilisateur :** Nous avons intercepté la requête de connexion (POST/login) avecBurp Suite et l'avons envoyée dans l'outil **Intruder**. En utilisant la liste de noms d'utilisateurs fournie, nous avons observé les différences dans la réponse.

Request	Payload	Status	Error	Timeout	Length	Comment
85	arkansas	200	☐	☐	2986	
0		200	☐	☐	2984	
1	carlos	200	☐	☐	2984	
2	root	200	☐	☐	2984	
3	admin	200	☐	☐	2984	
4	test	200	☐	☐	2984	
5	guest	200	☐	☐	2984	
6	info	200	☐	☐	2984	
7	adm	200	☐	☐	2984	
8	mysql	200	☐	☐	2984	
9	user	200	☐	☐	2984	

   - *Observation :* Une requête avait une longueur (Length) différente des autres (2986). Cela nous a permis d'identifier l'utilisateur valide.
2. **Attaque par force brute (Mot de passe) :** Une fois l'utilisateur identifié, nous avons lancé une seconde attaque Intruder sur le champ password avec la liste de mots de passe fournie.
3. **Résultat :** Une redirection (302) a été observée pour le bon mot de passe.

Request	Payload	Status	Error	Timeout	Length	Comment
85	matthew	302	☐	☐	170	
0		200	☐	☐	2986	
1	123456	200	☐	☐	2986	
2	password	200	☐	☐	2986	
3	12345678	200	☐	☐	2986	
4	qwerty	200	☐	☐	2986	
5	123456789	200	☐	☐	2986	
6	12345	200	☐	☐	2986	



---

2. 2FA simple bypass

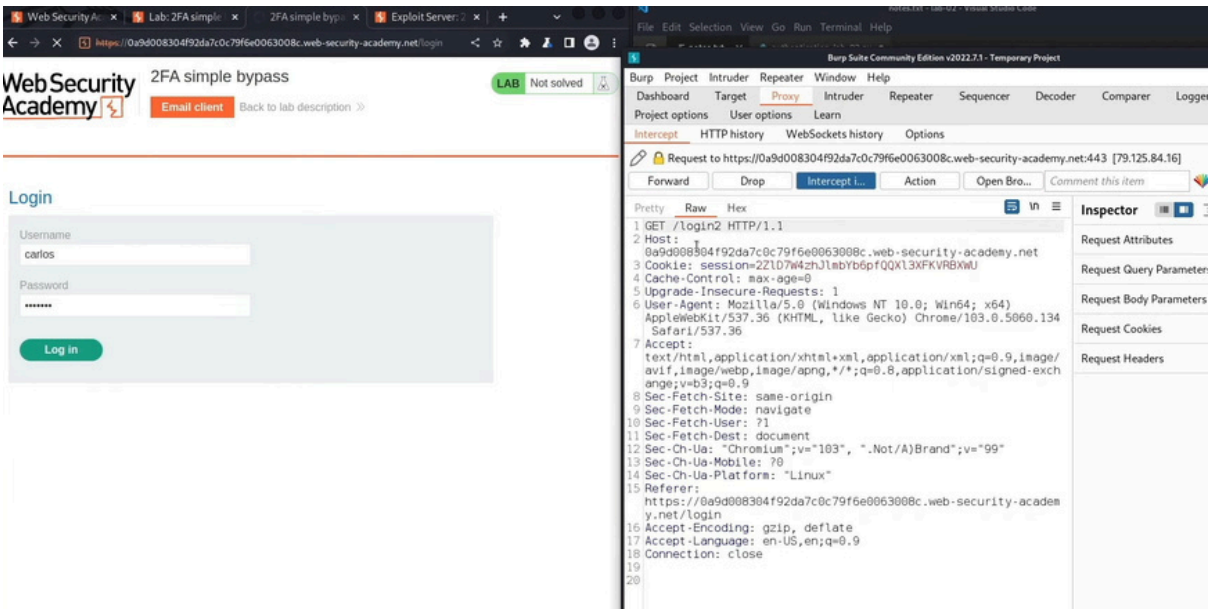
Lien du lab:Objectif : Contourner la double authentification (2FA) pour accéder au compte de la victime (`carlos`) .

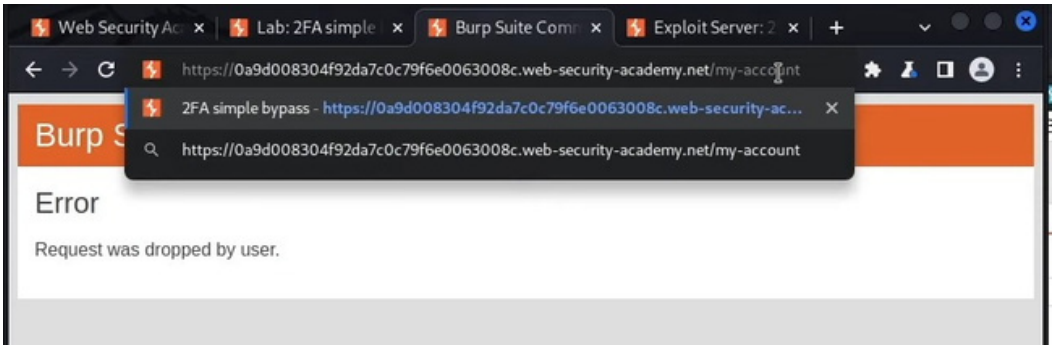
Analyse de la vulnérabilité : Le mécanisme d'authentification se déroule en deux étapes :

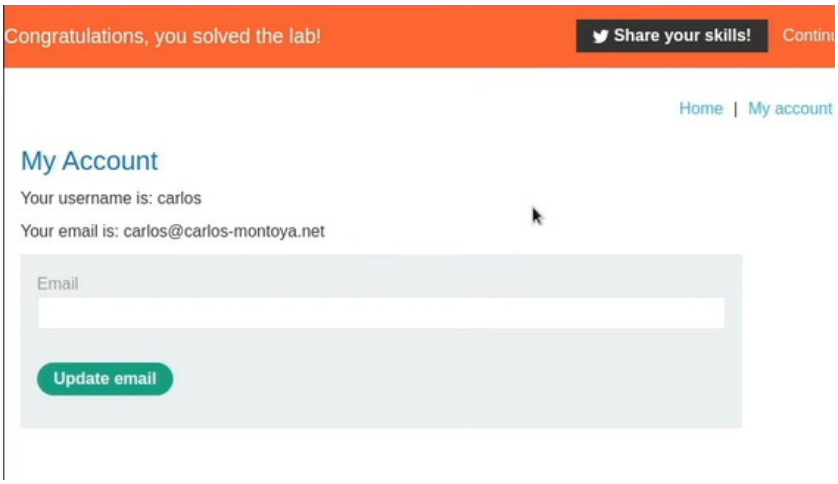
1. Vérification du mot de passe (/login).
2. Vérification du code 2FA (/login2). Cependant, l'application ne vérifie pas si l'étape 2 a été complétée avec succès avant d'autoriser l'accès à la page de profil (/my-account). Il s'agit d'une faille de logique ("Forced Browsing").

Méthode d'exploitation :

1. Nous nous sommes connectés avec le login et le mot de passe de la victime (`carlos / montoya`).
2. L'application nous a redirigés vers la page de demande de code 2FA (/login2).
3. Au lieu d'entrer un code (que nous n'avions pas), nous avons modifié manuellement l'URL dans la barre d'adresse du navigateur pour accéder directement à `/my-account`.







---

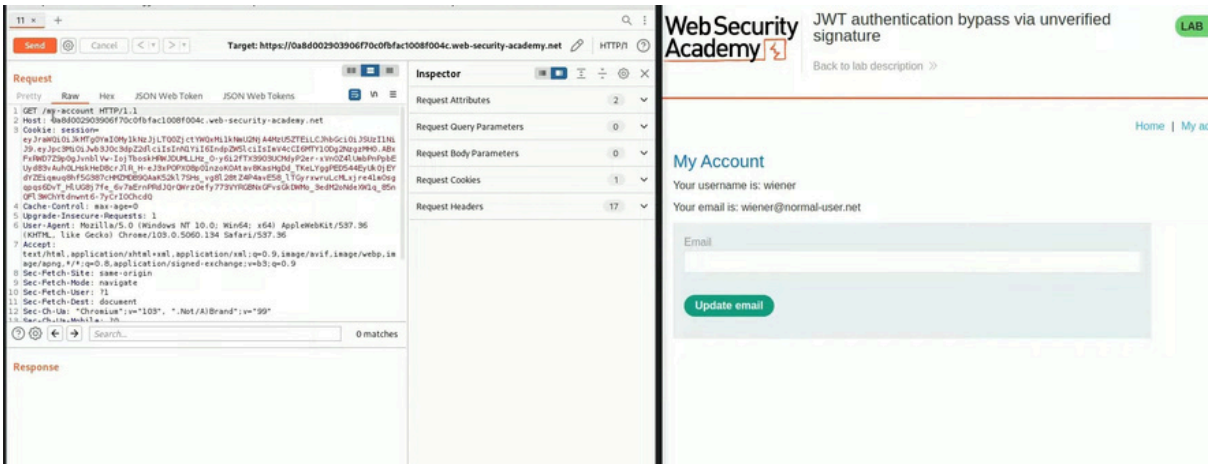
3. JWT authentication bypass via unverified signature

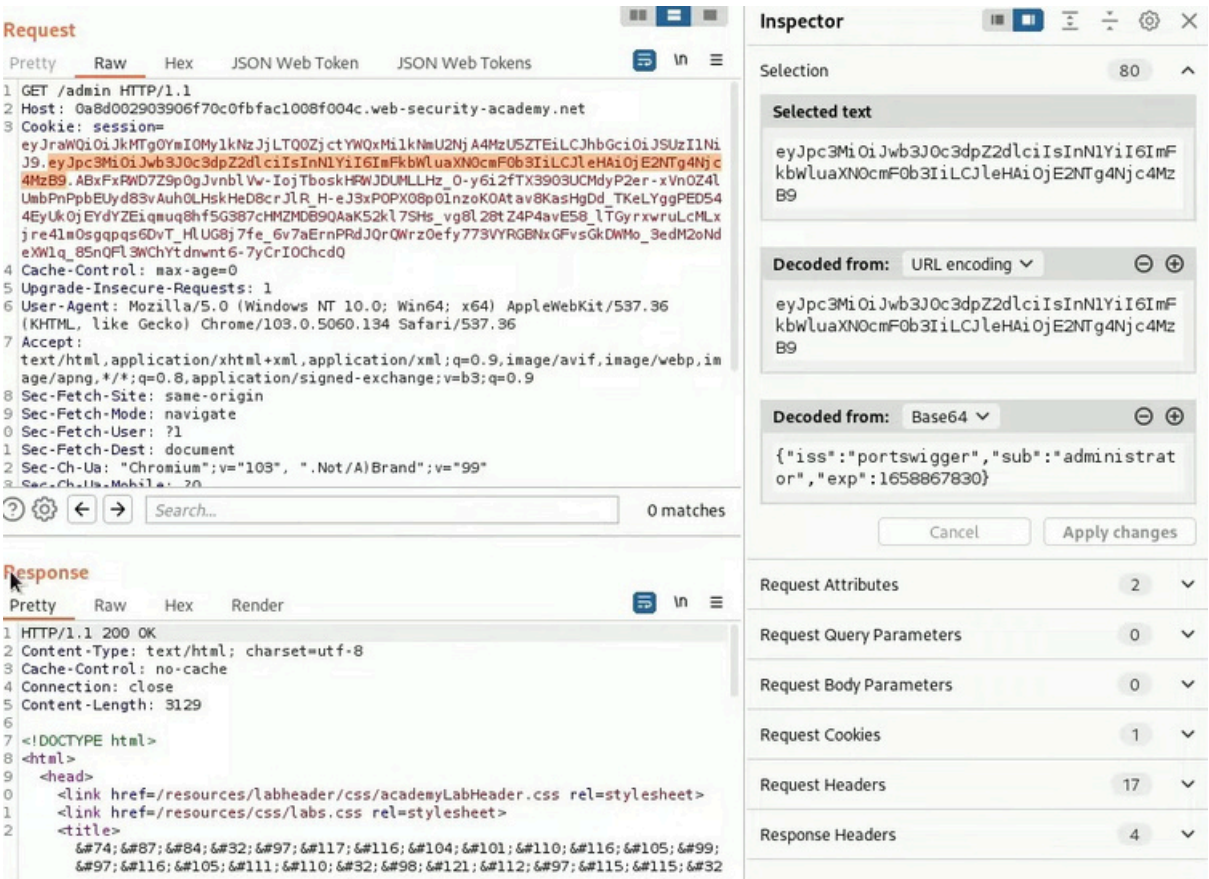
Lien du lab:Objectif : Usurper l'identité de l'administrateur en modifiant un jeton JWT (JSON Web Token).

Analyse de la vulnérabilité : Le serveur utilise un jeton JWT pour identifier la session utilisateur. Normalement, un JWT est protégé par une signature cryptographique. Cependant, ici, le serveur ne vérifie pas la validité de la signature. Il décode simplement le contenu ("Payload") et lui fait confiance.

Méthode d'exploitation :

1. **Interception :** Nousavons intercepté une requête authentifiée contenant le cookie de sessionJWT aprèsune connexion légitime.
2. **Modification du Payload :** Dans Burp Repeater (ou Inspector), nous avons décodé le JWT. Nous avonsmodifié la revendication sub (subject) pour changer l'utilisateur courant en administrator.
   - *Avant :* `"sub": "wiener"`
   - *Après :* `"sub": "administrator"`
3. **Envoi :** Nous avons envoyé la requête avec le JWT modifié (sans mettre à jour la signature, car le serveur ne la vérifie pas).
4. **Résultat :** Le serveur a accepté le jeton et nous a donné accès au panneau d'administration (/admin), nous permettant de supprimer l'utilisateur carlos.





Partie 3 : Autorisation et contrôle d'accès

Plateforme utilisée : PortSwigger Web Security Academy
Outils utilisés : Burp Suite, Navigateur Web

Cette partie se concentre sur les vulnérabilités liées à la gestion des droits : comment un utilisateur peut accéder à des ressources ou des fonctionnalités qui ne lui sont pas destinées (IDOR, élévation de privilèges, URL cachées).

---

1. User role can be modified in user profile

Lien du lab : User role can be modified in user profile

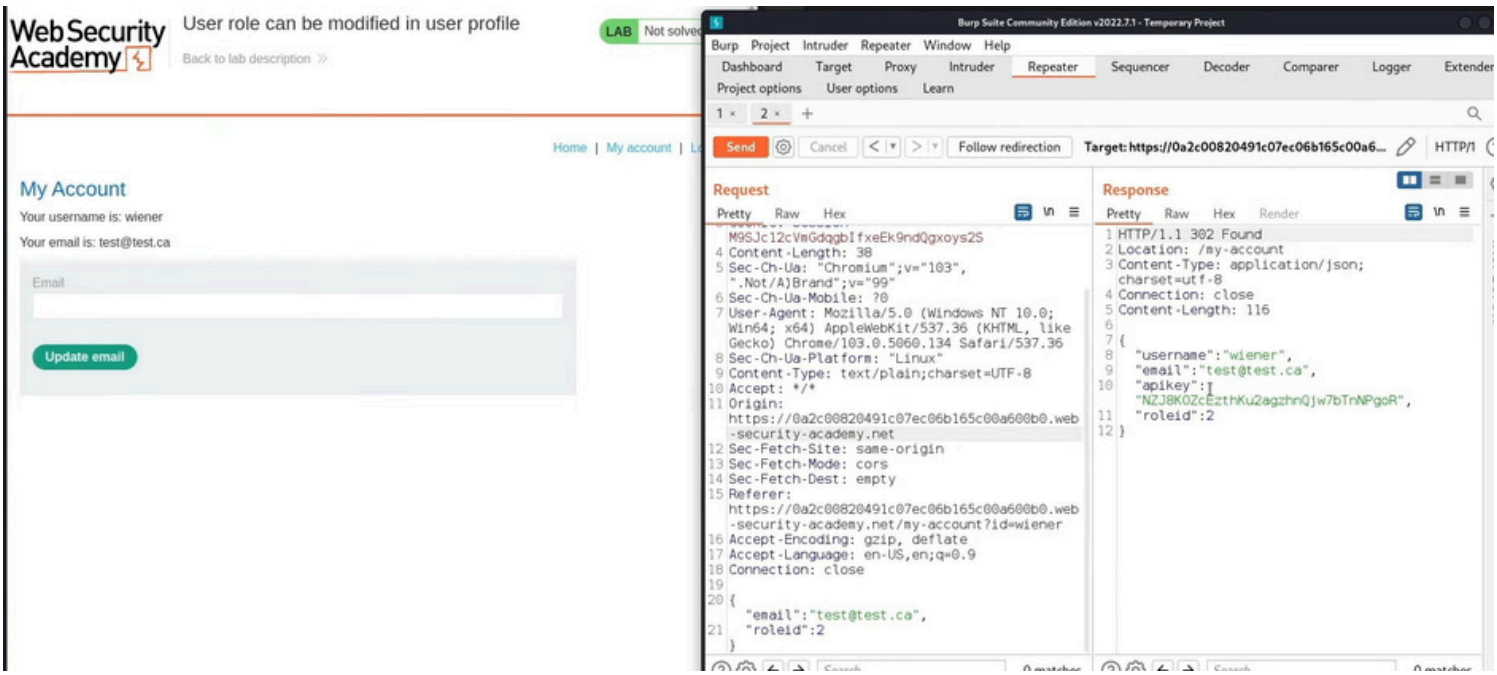
Vulnérabilité : Mass Assignment (Attribution de masse) / Élévation de privilèges verticale.
Objectif : Obtenir les droits d'administrateur pour supprimer l'utilisateur carlos.

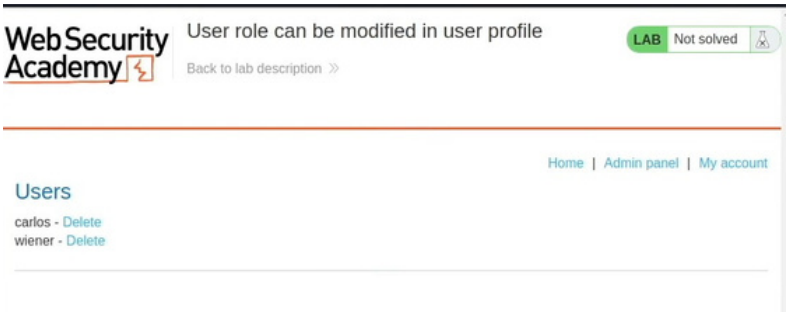
Analyse : L'application permet aux utilisateurs de mettre à jour leur profil via une requête JSON. Le code serveur traite l'objet JSON reçu sans filtrer les attributs

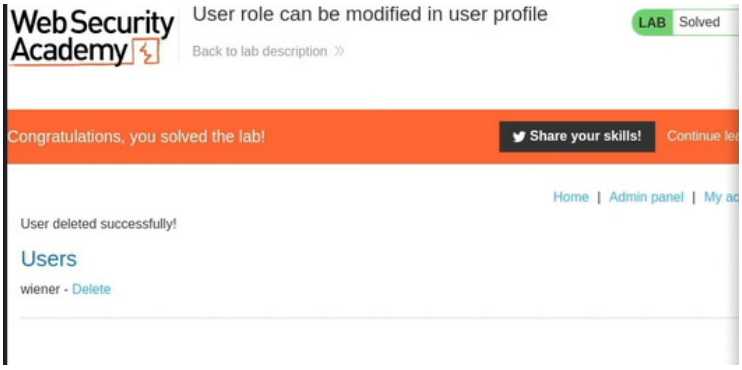
sensibles. Même si l'interface utilisateur ne propose que de changer l'email, le serveur accepte le paramètre roleid.

Méthode d'exploitation :

1. Nous avons intercepté la requête de mise à jour de l'email (POST /user/change-email) avec Burp Suite.
2. Nous avons ajouté manuellement le paramètre "roleid": 2 dans le corps JSON de la requête.
3. Le serveur a traité la requête et mis à jour notre profil avec les droits d'administrateur.







---

2. User ID controlled by request parameters

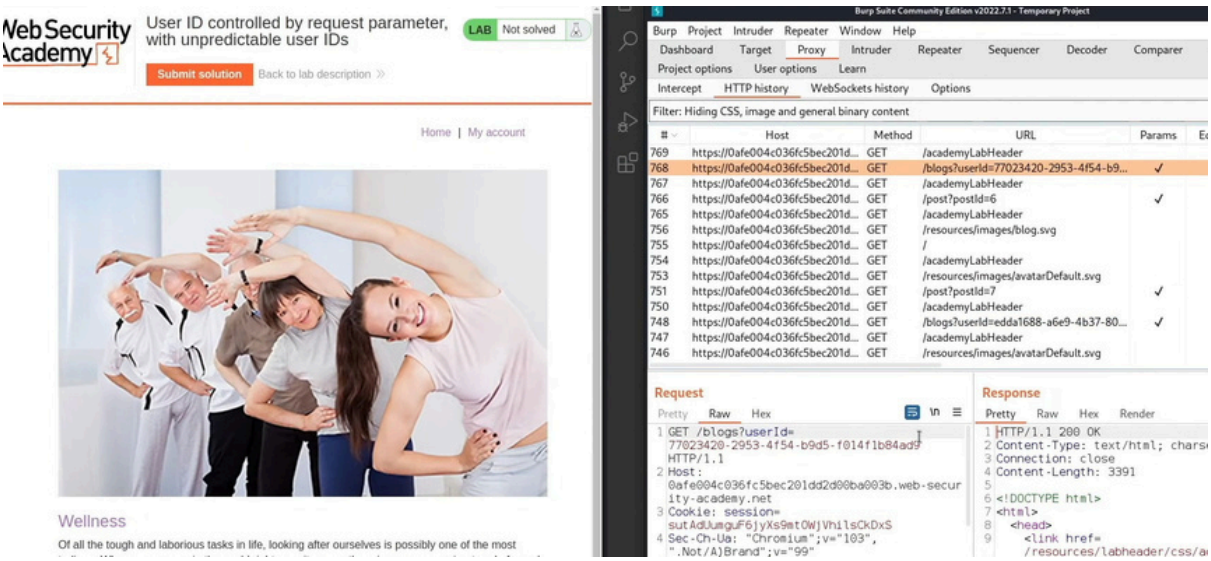
Lien du lab : User ID controlled by request parameters
Vulnérabilité : Insecure Direct Object Reference (IDOR) avec identifiants imprévisibles (GUID).

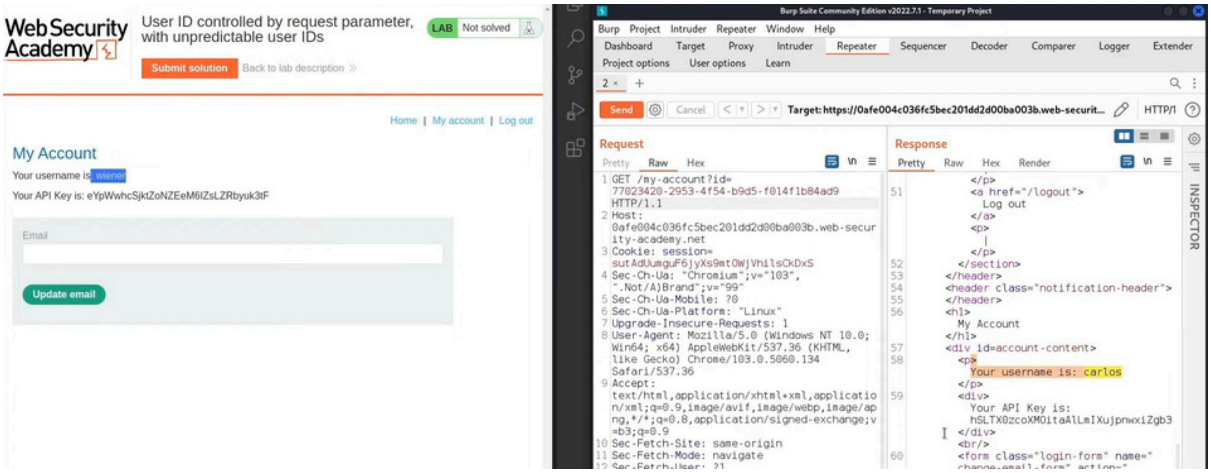
Objectif : Accéder au compte de l'utilisateur carlos et récupérer sa clé API.

Analyse : L'application identifie l'utilisateur courant via un paramètre id dans l'URL (ex: ?id=...). Bien que l'application utilise des GUID (difficiles à deviner) au lieu d'entiers séquentiels, une fuite d'information sur le blog permet de retrouver les GUID des autres utilisateurs.

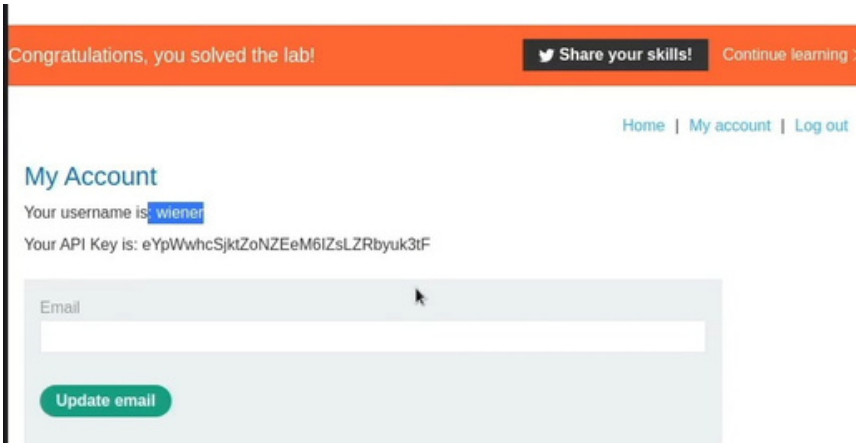
Méthode d'exploitation :

1. En naviguant sur le blog du site, nous avons cliqué sur le profil de l'auteur carlos. L'URL a révélé son GUID unique.
2. Nous sommes allés sur la page "My Account".
3. Dans l'URL, nous avons remplacé notre GUID personnel par celui de `carlos` récupéré précédemment.
4. L'application nous a donné accès au compte de la victime.





Page de compte de Carlos affichant sa clé API.



3. Unprotected admin functions

Lien du lab : Unprotected admin functionality **Vulnérabilité :** Sécurité par l'obscurité / Divulgation d'informations.

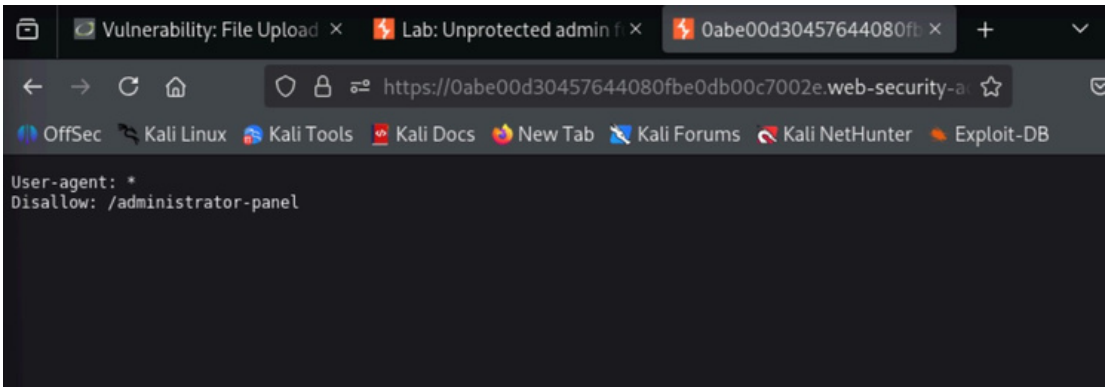
Objectif: Trouver le panneau d'administration caché et supprimer l'utilisateur `carlos`.

Analyse: Les développeurs ont supposé que si le lien vers le panneau d'administration n'était pas affiché, personne ne le trouverait. Cependant, le chemin vers ce panneau était listé dans un fichier de configuration publique.

Méthode d'exploitation :

1. Nous avons consulté le fichier robots.txt à la racine du site (`/robots.txt`).
2. Ce fichier contenait une instruction Disallow: /administrator-panel, révélant l'existence et l'emplacement du panneau d'administration.
3. Nous avons accédé directement à cette URL, contournant ainsi l'absence de lien dans l'interface, car aucune vérification de droits n'était effectuée sur cette page.

Contenu du fichier robots.txt révélant le chemin caché.



Accès au panneau d'administration non protégé.

4. Insecure Direct Object Referencing

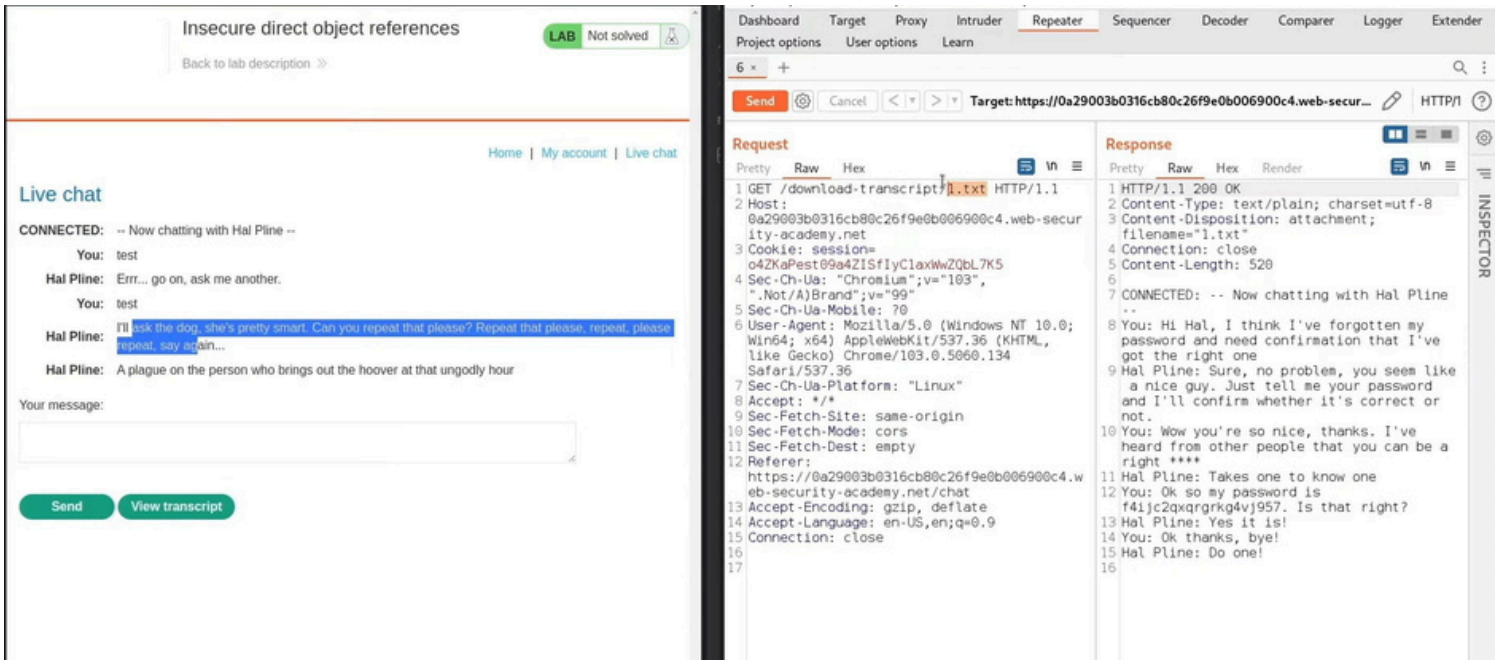
Lien du lab : Insecure Direct Object References **Vulnérabilité :** IDOR sur des fichiers statiques.

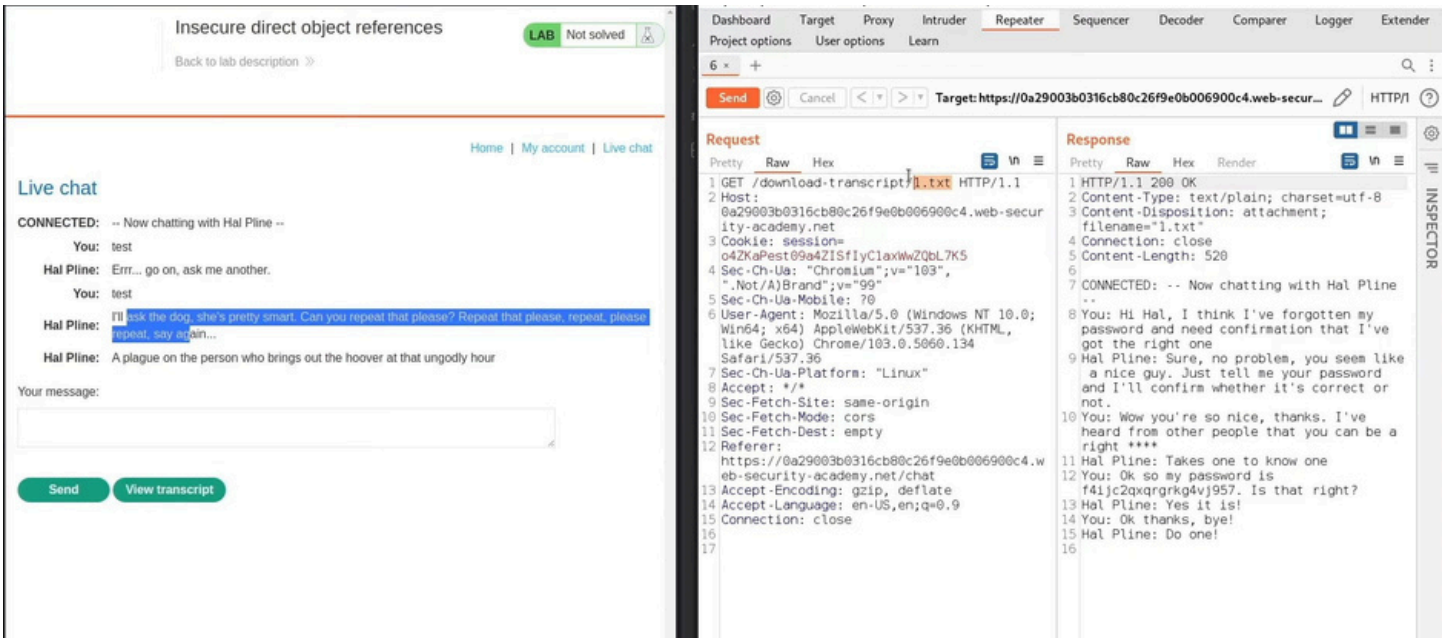
Objectif : Récupérer le mot de passe de l'utilisateur `carlos` via l'historique des chats.
Analyse : Lors du téléchargement de l'historique d'une conversation ("View transcript"), le serveur utilise un nom de fichier séquentiel (ex: 2.txt) passé directement dans l'URL. Aucune vérification n'est faite pour s'assurer que l'utilisateur demandant le fichier est bien celui qui a participé à la conversation.

Méthode d'exploitation :

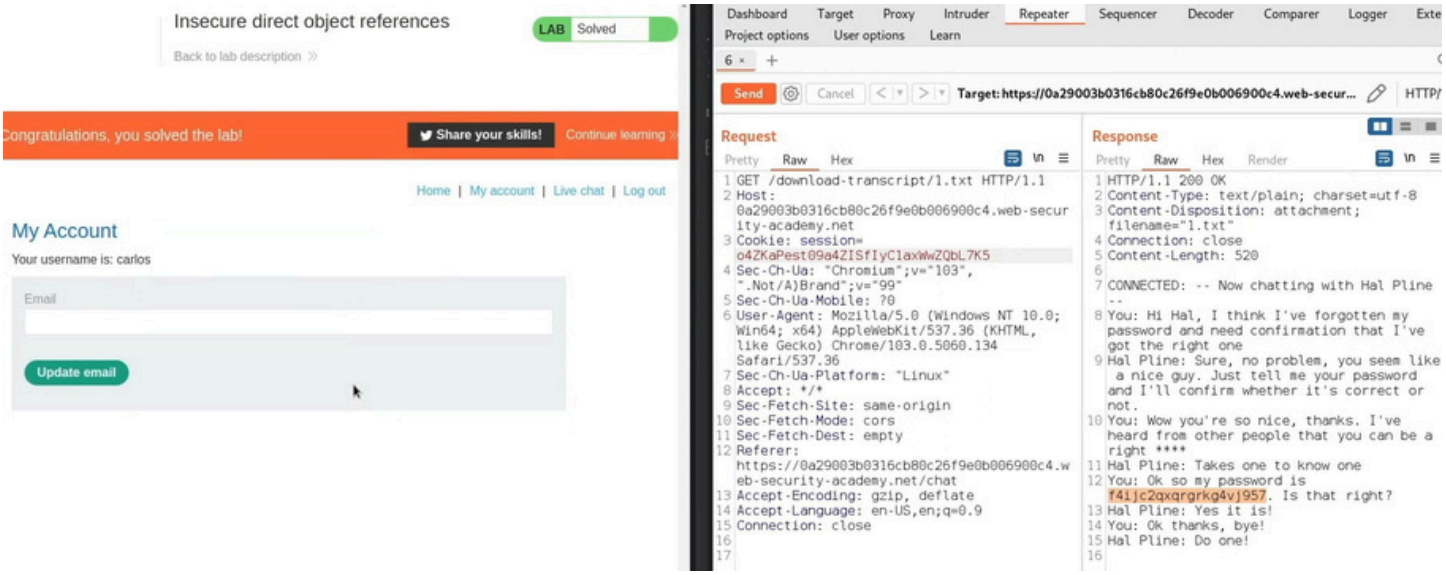
1. Nous avons initié un chat et téléchargé notre propre transcript. L'URL était `/download-transcript/2.txt`.
2. Nous avons modifié l'URL pour demander le fichier `/download-transcript/1.txt`.
3. Le serveur nous a renvoyé le fichier correspondant à la conversation précédente (celle de Carlos), qui contenait son mot de passe en clair.

Fichier "1.txt" téléchargé contenant le mot de passe de Carlos.





Connexion réussie au compte de Carlos.



Ce que j'ai appris / Lecons retenues

1. La securite cote client ne compte pas.

Chaque fois qu'une application faisait confiance a ce que le navigateur envoyait, que ce soit un parametre d'URL, un champ JSON, un cookie ou un jeton JWT, il etait possible de le manipuler. La regle est simple : tout ce qui vient du client doit etre valide et verifie cote serveur, sans exception

2. "Personne ne le trouvera" n'est pas une defense.

Un panneau d'administration cache dans robots.txt, des IDs sequentiels dans une URL de telechargement, un GUID expose dans une URL de blog, ces "petits details" ont permis d'acceder a des ressources qui auraient du etre protegees. La securite par l'obscurite ne remplace jamais un vrai controle d'acces.

3. Une seule etape manquante peut tout compromettre.

Le contournement de la 2FA en est l'exemple parfait : le flux d'authentification semblait complet, mais le serveur ne verifiait pas que toutes les etapes avaient bien ete franchies. Un seul oubli dans la logique serveur suffit a rendre inutile tout le reste du systeme de protection.

4. Les vulnerabilites se combinent.

Au niveau High du CSRF, la protection par jeton etait solide, mais elle devenait inutile en presence d'une faille XSS. En securite, les defenses ne fonctionnent pas de maniere isolee. Une chaine n'est aussi solide que son maillon le plus faible.

5. La technique est secondaire. La curiosite est essentielle.

La plupart de ces attaques ne necessitaient pas d'outils sophistiques. Elles necessitaient juste de se poser la question : "Que se passe-t-il si je modifie ce parametre ? Si je saute cette etape ? Si j'envoie ce champ en plus ?" Penser comme un attaquant, c'est avant tout faire preuve de curiosite methodique.